MP1 Project Report

Project 1: Small Language Model Challenge

Student ID: 3036779970

Student Name: Zhang Yiyi

Course: DASE7506

1. Introduction

The objective of this project is to improve the baseline GPT language model provided by the course framework while satisfying the evaluation constraints specified in the assignment. The evaluation metric is **Bits Per Byte (BPB)** on the WikiText-2 benchmark, where lower BPB indicates better language modeling performance. The baseline model achieves approximately **2.10 BPB** on the test set.

Instead of simply increasing model size, this project modernizes both the model architecture and the training procedure. Several improvements from recent Transformer literature are incorporated while maintaining strict causal decoding and the evaluation constraints. The final model combines:

1) Rotary Position Embedding (RoPE)
2) RMSNorm
3) SwiGLU Feed-Forward Network
4) Increased model capacity
5) Dropout regularization
6) Pointer Sentinel Cache
7) Improved optimization schedule
8) Exponential Moving Average (EMA)

These modifications substantially improve the prediction accuracy while remaining reproducible under the provided evaluation framework.

2. Baseline Model

The provided baseline model is a standard GPT implementation consisting of:

1) 4 Transformer blocks
2) Hidden dimension of 128
3) 4 attention heads
4) Learned absolute positional embeddings
5) LayerNorm
6) GELU activation
7) AdamW optimizer
8) Fixed learning rate schedule

The baseline contains approximately **1.09 million parameters** and achieves approximately:

Model	**Test BPB**
Baseline GPT	**2.1013**

Although simple and efficient, several limitations were identified:

1) Learned positional embeddings generalize poorly under limited training data.
2) LayerNorm is computationally heavier than RMSNorm.
3) GELU feed-forward networks are less expressive than SwiGLU.
4) Small model capacity limits representation ability.
5) No mechanism exists for copying recently observed tokens.
6) Training does not exploit modern optimization techniques such as EMA.

3. Proposed Improvements

3.1 Larger Transformer Architecture

The baseline model (4 layers × 128 hidden units) was expanded to:

- Hidden dimension: 256
- Transformer layers: 6
- Attention heads: 4

The increased capacity enables the model to learn richer semantic and syntactic representations from the training corpus while remaining within the evaluation budget.

3.2 Rotary Position Embedding (RoPE)

The baseline employs learned absolute positional embeddings.

These embeddings are replaced with **Rotary Position Embeddings (RoPE)**.

Instead of learning position vectors, RoPE rotates the query and key vectors inside the attention mechanism according to token position.

Advantages include:

- Better extrapolation of positional information
- More stable optimization
- Improved modeling of relative token distances
- No additional trainable positional parameters

Implementation highlights:

```
q, k = apply_rope(q, *rope), apply_rope(k, *rope)
```

This change removes the learned positional embedding table entirely.

3.3 RMSNorm

LayerNorm is replaced by RMSNorm.

Compared with LayerNorm:

- lower computational overhead
- simpler normalization
- improved numerical stability
- widely adopted in recent large language models (e.g., LLaMA)

Implementation:

```
self.norm1 = RMSNorm(width)
self.norm2 = RMSNorm(width)
```

3.4 SwiGLU Feed-Forward Network

The baseline uses **Linear → GELU → Linear .**

The proposed model replaces it with

```
Linear
      ├─ SiLU(gate)
      └─ value
gate x value
Linear
```

Advantages:

- richer nonlinear interactions
- improved gradient propagation
- higher parameter efficiency

Implementation:

```
gate, value = h.chunk(2)
h = F.silu(gate) * value
```

3.5 Dropout Regularization

Residual dropout and embedding dropout are introduced.

Purpose:

1) reduce overfitting
2) improve generalization
3) stabilize deeper models

Dropout is automatically disabled during evaluation.

3.6 Pointer Sentinel Cache

One of the major architectural improvements is an optional Pointer Sentinel mechanism.

Instead of predicting every token solely from the vocabulary distribution, the model can copy recently observed tokens from the current context window.

The final probability is

$$P(v)=a_{null}P_{LM}(v)+\sum_{j} a_j$$

where

- P_{LM} is the language model prediction
- a_j are attention weights over previous tokens
- a_{null} preserves the original language model distribution.

Advantages:

- improves prediction of repeated entities
- better modeling of names and rare words
- no external memory
- satisfies the causal evaluation constraint

Implementation:

```
mixture = (
    torch.softmax(logits.float(), dim=-1)
    * keep_lm
).scatter_add(...)
```

4. Training Improvements

Besides architectural changes, the training recipe was also modernized.

4.1 Cosine Learning Rate Decay

The baseline learning rate schedule was replaced by

- Warmup
- cosine decay

The learning rate is

$$LR = LR_{max} * warmup * cosine$$

Advantages:

- smoother convergence
- avoids instability during early training
- improves final optimization

4.2 Parameter-wise Weight Decay

Weight decay is applied only to weight matrices.

Biases and normalization parameters receive **no weight decay**.

This follows common practice in modern Transformer training.

4.3 Exponential Moving Average (EMA)

An EMA copy of model parameters is maintained during training.

Instead of using the final weights directly, validation is performed on both

- raw model
- EMA model

The checkpoint with the lowest validation BPB is automatically selected.

Advantages:

- smoother optimization
- better generalization
- reduced variance

4.4 Validation-Based Model Selection

Unlike the baseline, which simply saves the final model, the modified trainer:

- periodically evaluates validation BPB
- compares raw and EMA weights
- stores the best-performing checkpoint

This strictly follows the assignment rule that the test set must never be used for model selection.

5. Experimental Setup

Dataset:

- WikiText-2
- Fixed BPE-2048 tokenizer

Evaluation:

- FP32
- CPU
- official evaluator

Training:

```
6000 steps
batch size = 64
learning rate = 1.5e-3
EMA = 0.999
warmup
cosine decay
```

6. Experimental Results

6.1 Validation Performance

Model	Validation BPB	Token PPL
Final Model	**1.4866**	**23.13**
No Pointer	1.5419	25.99
No RoPE	1.5203	24.84

The final model achieves the lowest validation BPB among all configurations.

Compared with the ablation models:

- removing Pointer increases BPB by 0.0553
- removing RoPE increases BPB by 0.0337

Both components contribute positively to language modeling performance.

6.2 Test Performance

Model	Test BPB	Token PPL	CPU Time (s)
Baseline	**2.1013**	80.85	8.50
Final Model	**1.5035**	23.17	29.13

Compared with the baseline:

- BPB decreases from 2.1013 to 1.5035
- absolute improvement: 0.5977 BPB
- relative improvement: 28.4%
- token perplexity decreases by approximately 71%

Although inference time increases, the evaluation time remains within the assignment limit (29.13 s vs. 5 × 8.50 s = 42.5 s).

7. Ablation Study

Two controlled experiments were performed.

Removing Pointer Cache

Model	BPB
Final	**1.4866**
No Pointer	1.5419

Performance degrades noticeably after removing the pointer mechanism.

This demonstrates that copying recently observed tokens significantly improves prediction accuracy, particularly for repeated words and named entities.

Removing RoPE

Model	BPB
Final	**1.4866**
No Rope	1.5203

RoPE also contributes to lower BPB.

Relative positional encoding enables more effective attention over long contexts than learned positional embeddings.

8. Discussion

The improvements can be divided into two categories.

Architectural improvements

- RoPE
- RMSNorm
- SwiGLU
- Larger Transformer
- Pointer Cache

These directly improve the representational power of the model.

Training improvements

- EMA
- Cosine LR
- Warmup
- Better weight decay
- Validation checkpoint selection

These improve optimization stability and generalization.

Among all modifications, the Pointer Cache contributes the largest individual improvement in the ablation study.

9. Limitations

Despite achieving substantial improvements, several limitations remain.

1) The larger model increases inference latency.
2) Pointer Cache introduces additional computation during evaluation.

3) Only WikiText-2 was used for training due to assignment constraints.
4) Hyperparameter search was limited by available computational resources.

Future work could explore:

- FlashAttention
- grouped-query attention
- Mixture-of-Experts
- better learning-rate schedules
- longer context lengths

10. Conclusion

This project modernizes the provided GPT baseline by incorporating several techniques widely adopted in recent Transformer architectures.

The final model combines:

- larger Transformer architecture
- Rotary Position Embeddings
- RMSNorm
- SwiGLU
- Dropout
- Pointer Sentinel Cache
- Cosine learning-rate schedule
- Exponential Moving Average

Experimental results demonstrate significant improvements over the baseline.

The final model achieves:

- **Validation BPB: 1.4866**
- **Test BPB: 1.5035**

compared with the baseline **2.1013 BPB**, corresponding to an absolute improvement of **0.5977 BPB (28.4%)** while remaining within the required evaluation constraints.